UNIVERSITÉ DE LORRAINE | IUT Saint-Dié-des-Vosges

# Proposition technique

## Outil de gestion commerciale des Jardins de Cocagne

Software Architecture Évolutions × Réseau Cocagne

**Timothé BELCOUR – Abdoulaye KABA**

BUT Informatique 3e année – SAÉ 5 Développement avancé – Rendu 3
IUT de Saint-Dié-des-Vosges – Université de Lorraine
Année universitaire 2026-2027 · 15 octobre 2026

# Sommaire


I

## Le besoin

Ce que le Réseau Cocagne attend

II

## L'architecture

Comment la solution est organisée

III

## Les technologies

Nos choix et pourquoi

IV

## Les fonctions clés

Carte, itinéraire, calendrier, composition des paniers

V

## Déploiement et organisation

Sécurité, hébergement, conduite du projet

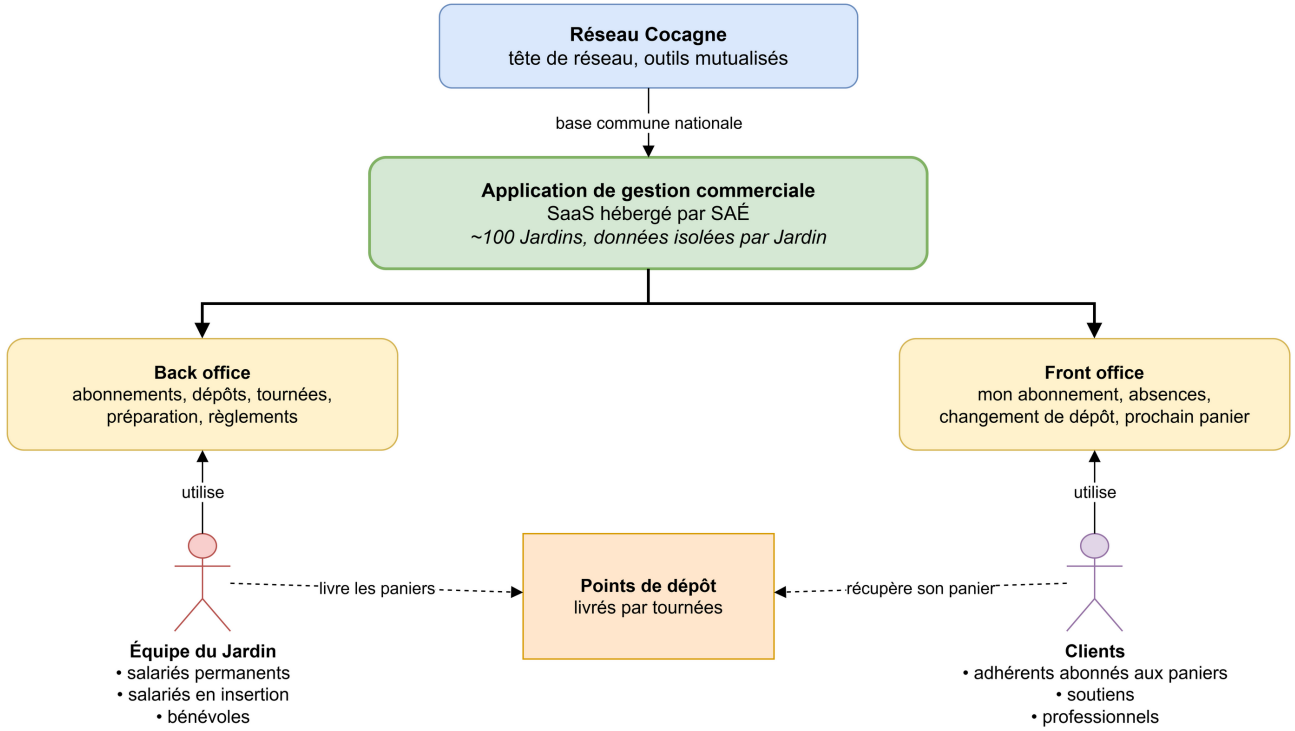

*Chaque choix technique répond à une exigence du cahier des charges.*

I

# Un outil commun pour près de 100 Jardins

**75 %** des Jardins ont un besoin urgent d'un nouvel outil (logiciel Access des années 2000)

- **Objectif** : un outil mutualisé, adapté au fonctionnement de chaque Jardin
- **Back office** : abonnements, dépôts, tournées, préparation, règlements
- **Front office** : abonnement, absences, changement de dépôt, prochain panier
- **Défi** : une grande diversité de fonctionnements → un paramétrage poussé



I

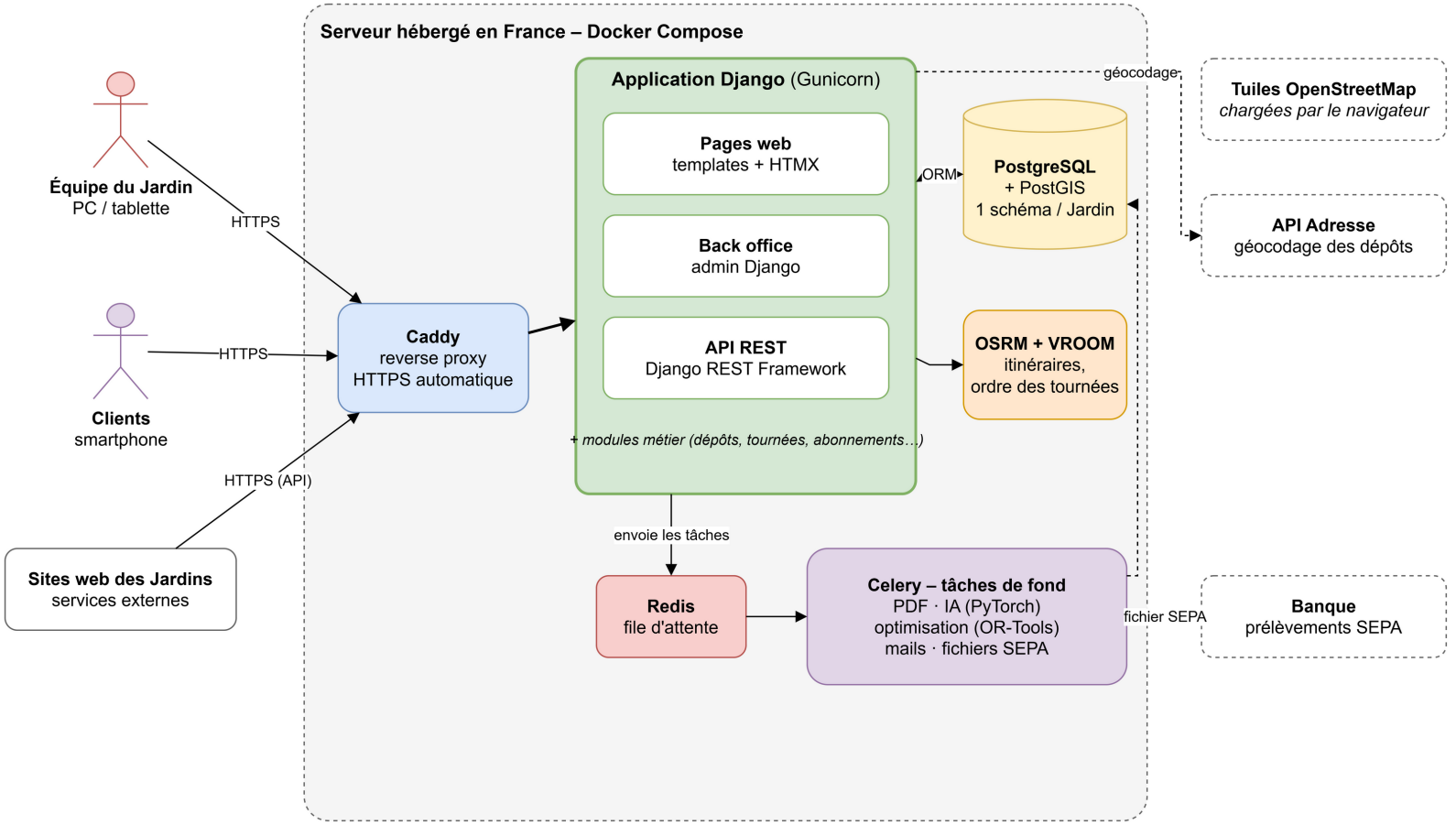
# Exigences du Réseau Cocagne → nos réponses

| Exigence du client | Notre réponse |
|---|---|
| **Logiciel libre (open source)** | Uniquement des composants libres, code publié sous licence libre |
| **Mode SaaS, hébergement centralisé et sécurisé** | Une seule application hébergée et maintenue par SAÉ, mise à jour pour tous en une fois |
| **Multi-structures, les données restent au Jardin** | Données isolées par Jardin, export possible à tout moment |
| **Modules activables au choix** | Architecture modulaire : chaque Jardin active ce qu'il utilise |
| **Très haut niveau de paramétrage** | Saisons, fréquences, tarifs, catégories, calendriers paramétrables |
| **Utilisateurs peu à l'aise avec l'informatique** | Interface simple, info-bulles, aide en ligne, tutoriels vidéo |
| **Langage simple, contributions de bénévoles** | Python / Django : langage le plus enseigné, code commenté et documenté |
| **Légalité : RGPD, loi de finances, SEPA** | Conformité prise en compte dès la conception |

*Chaque choix technique présenté ensuite découle de ces exigences.*

II

# Comment fonctionne la solution

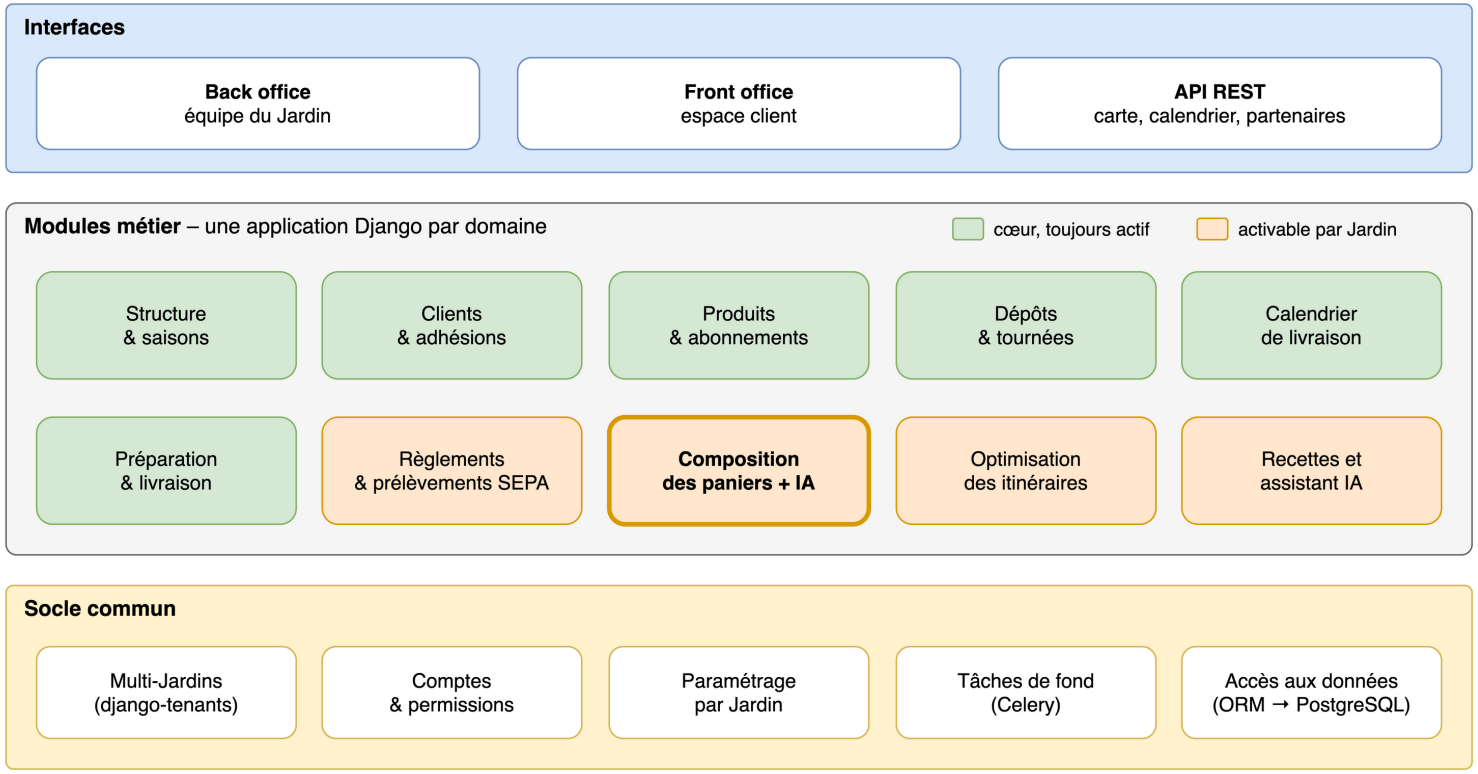



- **Hébergée en France** : une seule application web
- **Caddy** : porte d'entrée, HTTPS automatique
- **Django** : pages, back office et API
- **Celery** : tâches longues en arrière-plan (PDF, IA, calculs)
- **Docker** : installation identique partout

II

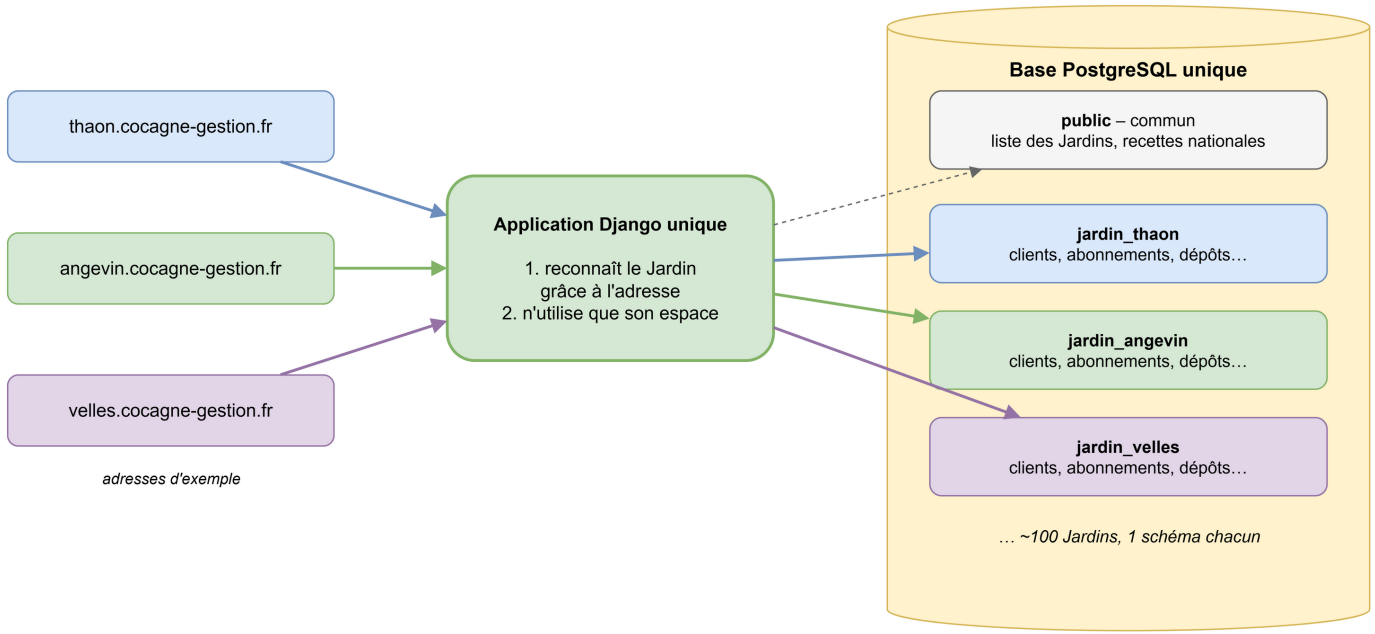
# Une application découpée en modules

- **1 module = 1 domaine** du métier
- **Cœur** (vert) : toujours actif
- **Modules activables** (orange) : chaque Jardin choisit
- **Socle commun** : multi-Jardins, droits, paramétrage
- **Évolutif** : ajouter un module sans toucher au reste, les bénévoles peuvent contribuer



II

# Un seul logiciel, chaque Jardin chez soi




- **Une adresse web** propre à chaque Jardin
- **Un espace séparé** par Jardin, dans une seule base
- **Isolation** : un Jardin ne voit jamais les données d'un autre
- **Données du Jardin** : export ou suppression à tout moment
- **Une mise à jour** = tous les Jardins à jour

| Solution | Isolation des données | Maintenance |
|---|---|---|
| 1 base par Jardin | ● forte | ● ~100 bases à gérer |
| 1 base, tables partagées | ● risque de mélange | ● simple |
| 1 base, 1 schéma par Jardin (retenu) | ● **forte** | ● **une seule base** |

III

# Quel framework choisir ?

| Critère du client | Django (Python) | Laravel (PHP) | Symfony (PHP) | NestJS + React | Spring Boot |
|---|---|---|---|---|---|
| Open source | ● **oui** | ● oui | ● oui | ● oui | ● oui |
| Langage simple pour des bénévoles | ● **Python** | ● PHP | ● plus long à apprendre | ● 2 langages | ● lourd |
| Back office rapide à construire | ● **admin intégrée** | ● paquet externe | ● paquet externe | ● tout à coder | ● tout à coder |
| Plusieurs Jardins sur une application | ● **django-tenants** | ● paquet tenancy | ● à coder | ● à coder | ● à coder |
| Un seul langage avec l'IA | ● **oui** | ● non | ● non | ● non | ● non |
| Maintenance longue durée | ● **LTS 3 ans** | ● 2 ans | ● LTS | ● variable | ● LTS |

**Retenu : Django**

**Alternative crédible : Laravel**

● adapté ● partiel ● inadapté

*PHP et Dolibarr n'étaient que des pistes du client : son critère est un langage simple, et Python est le plus enseigné ; Dolibarr est un ERP généraliste où abonnements, tournées et composition des paniers resteraient à développer en modules.*

PARTIE III – LES TECHNOLOGIES

III

# Python et Django

**Python 3.13**
langage lisible, référence de l'IA

**Django 5.2 LTS**
framework « tout inclus », maintenu jusqu'en 2028

**DRF**
Django REST Framework : l'API

**HTMX**
pages interactives sans JavaScript lourd

**Bootstrap 5**
interface responsive et accessible

## 3 raisons

### 1 Simple

Des bénévoles des Jardins peuvent reprendre le code. Python est le langage le plus enseigné.

### 2 Complet

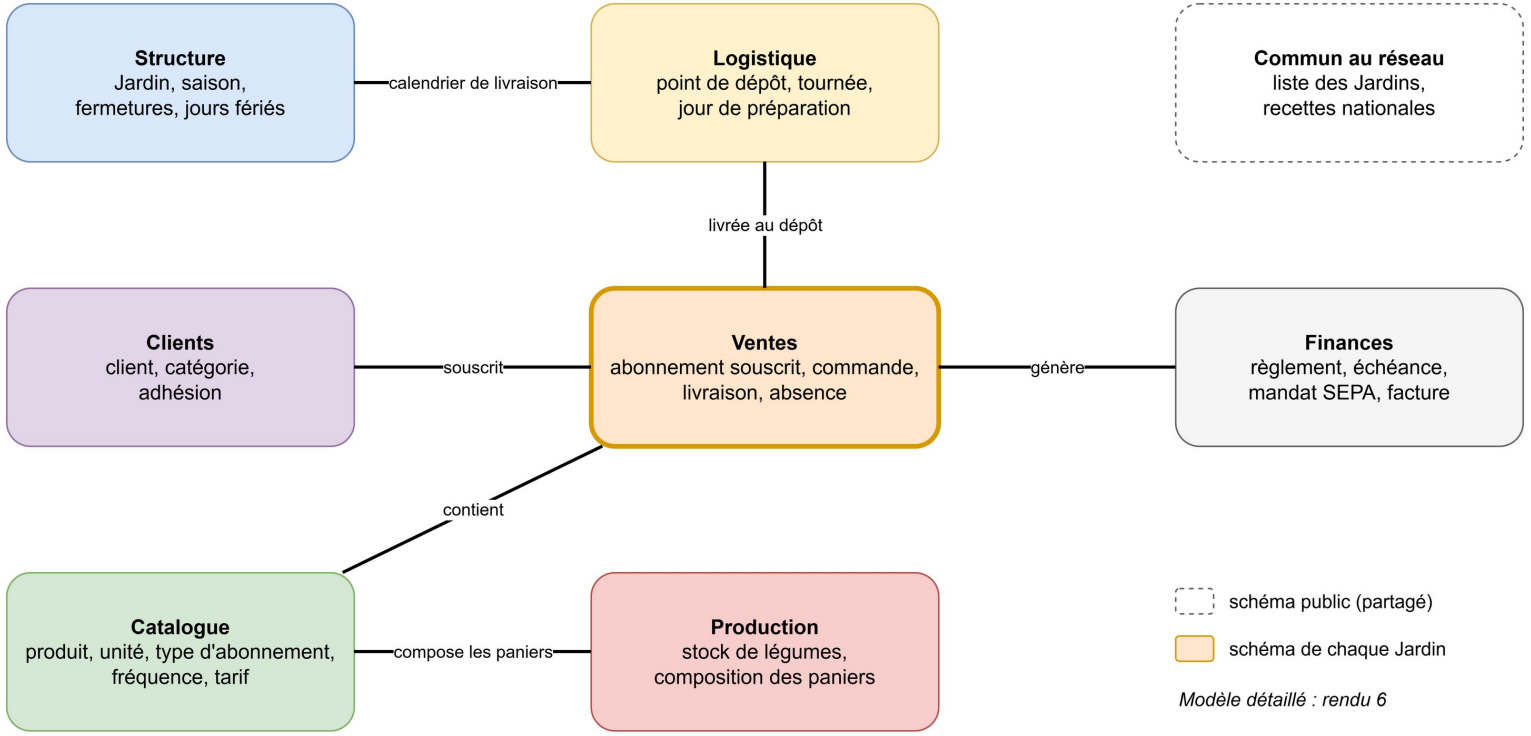
Admin, comptes, sécurité et base de données déjà inclus : moins de code à écrire et à maintenir.

### 3 Durable

Version LTS maintenue jusqu'en 2028, très grande communauté (Instagram, Mozilla…).

III

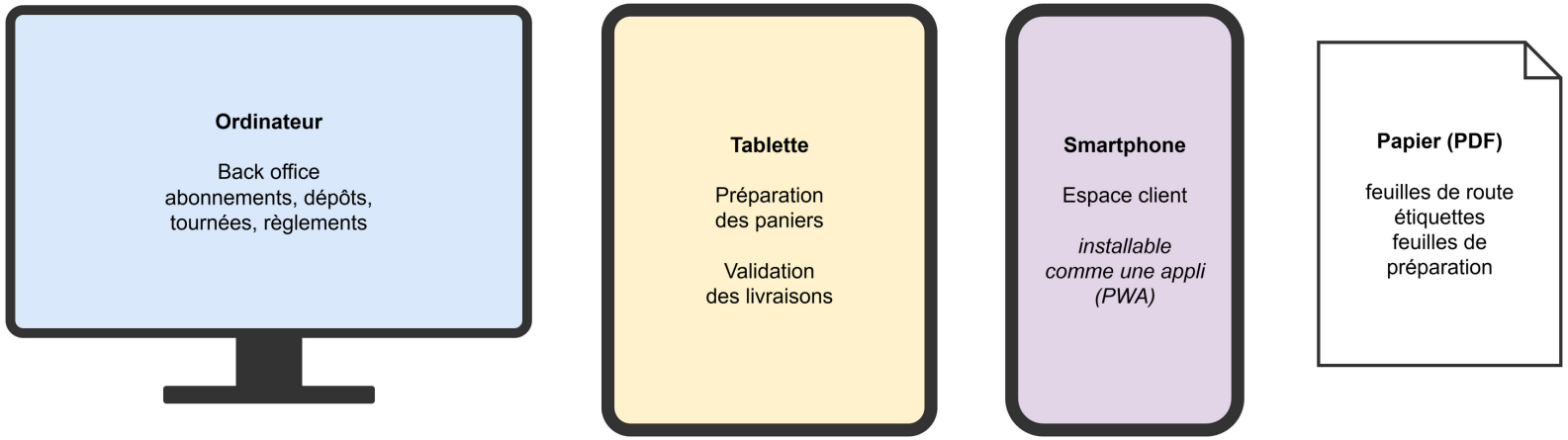
# Les données




- **PostgreSQL 17** : base libre et robuste
- **PostGIS** : positions GPS des dépôts, distances
- **Un schéma par Jardin** : données séparées
- **ORM Django** : des objets Python, pas de SQL à écrire
- **Migrations** : la base évolue avec le code

*Vue d'ensemble – le modèle de données détaillé fait l'objet du rendu 6*

# Une application, tous les écrans



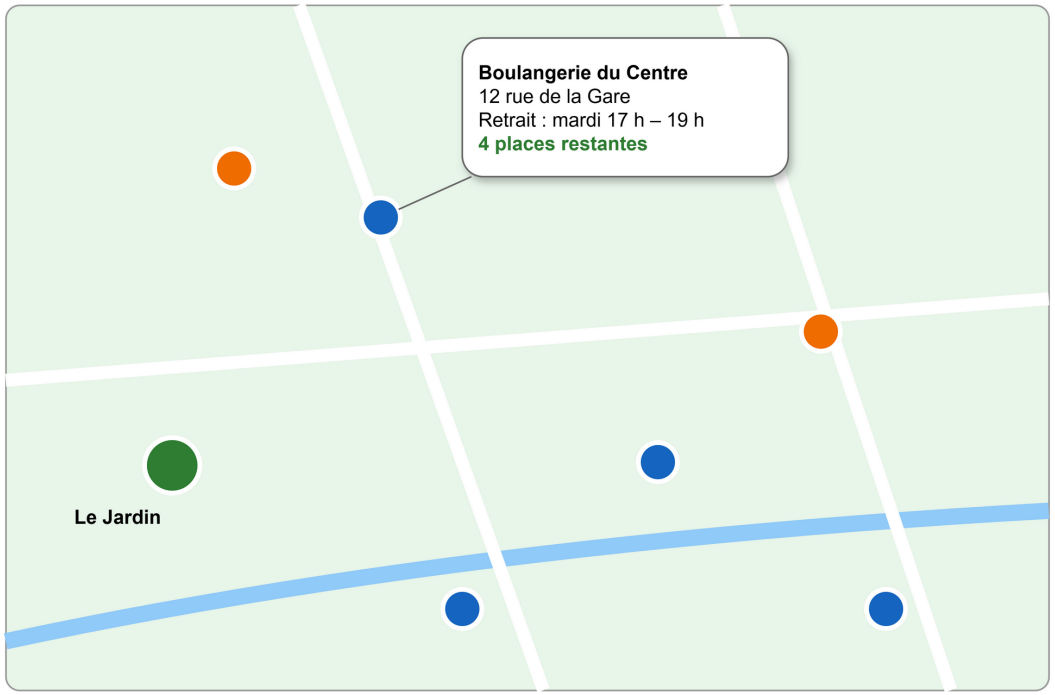

**Une seule application web responsive** (Bootstrap 5) – un navigateur suffit, rien à installer

- **Ordinateur** : gestion (back office)
- **Tablette** : préparation des paniers, validation des livraisons
- **Smartphone** : espace client, installable comme une appli, sans store
- **Papier** : feuilles de route et étiquettes en PDF
- **Tout est accessible partout** : ici, l'usage principal de chaque écran

**PWA (Progressive Web App)**
Un site web installable sur l'écran d'accueil comme une application, qui fonctionne en partie hors connexion.

IV

# La carte des points de dépôt




**Légende**

- le Jardin
- dépôt ouvert à tous
- dépôt réservé (mot de passe)

*Dépôts professionnels : non affichés sur la carte publique*

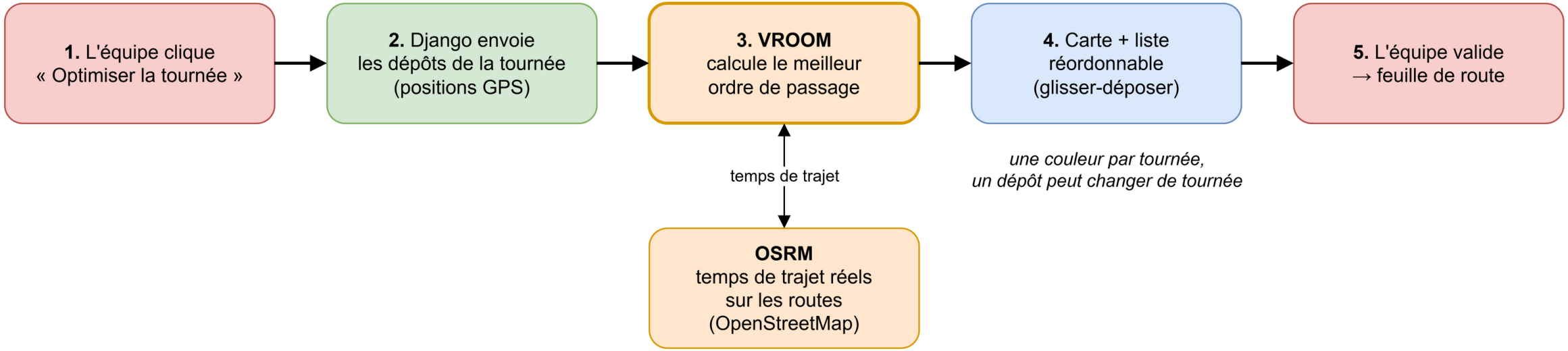
Fond : OpenStreetMap
Affichage : Leaflet
Positions : API Adresse

- **Leaflet + OpenStreetMap** : libres et gratuits
- **API Adresse** : adresse → position GPS automatique
- **Une couleur par catégorie** de dépôt
- **Dépôts professionnels** masqués sur la carte publique
- **Au clic** : horaires, places restantes ou COMPLET

**Pourquoi pas Google Maps ?** Payant, propriétaire, clé d'API obligatoire : contraire à l'exigence open source.

IV

# Les tournées de livraison



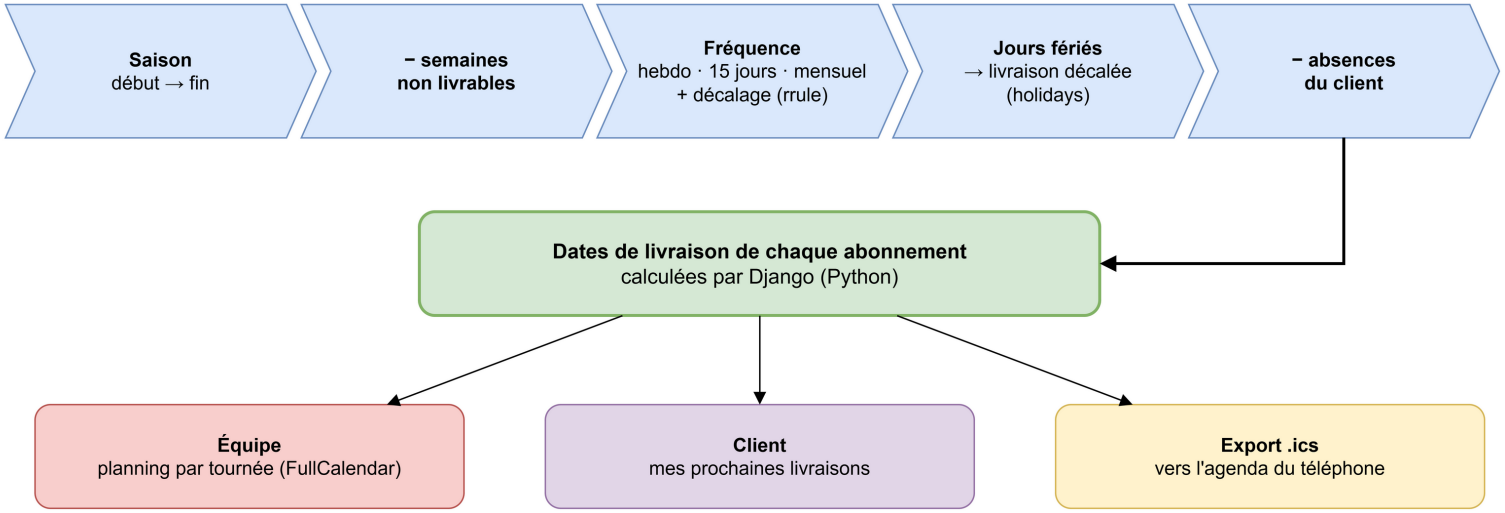

- **OSRM** : trajet réel sur les routes (distance, durée)
- **VROOM** : meilleur ordre de passage des dépôts
- **L'équipe garde la main** : l'outil propose, l'humain valide

- **Glisser-déposer** : réordonner une tournée, changer un dépôt de tournée
- **Auto-hébergé** : pas de quota, pas de coût à l'usage

*Pourquoi pas Google Directions ? Payant à l'usage, propriétaire.*

IV

# Le calendrier de livraison




- **Automatique** : dates calculées pour chaque abonnement
- **Tient compte** de la saison, des fermetures, de la fréquence, des jours fériés et des absences
- **Lissage** : paniers mensuels répartis sur plusieurs semaines
- **FullCalendar** : affichage pour l'équipe et les clients
- **Export .ics** vers l'agenda du téléphone

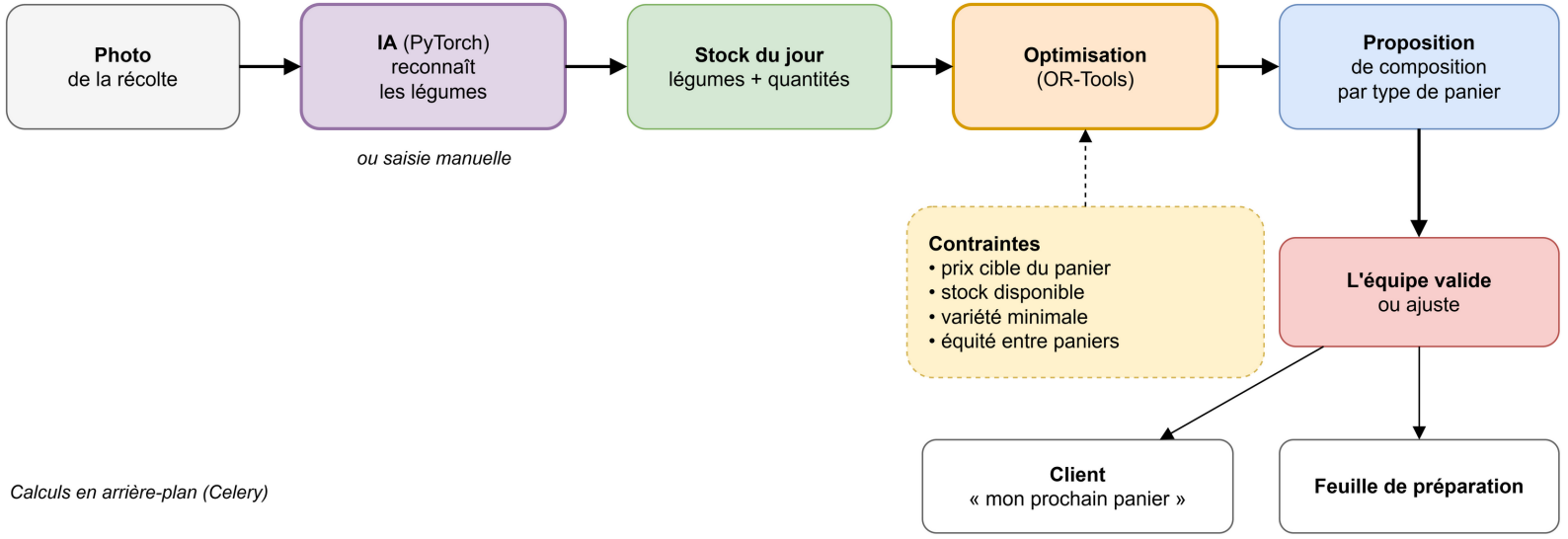
**Exemple (cahier des charges)**
Mardi 8 mai férié → livraison décalée au mercredi 9 mai

IV

# Composer des paniers équitables

Notre module

**Le problème :** répartir la récolte entre les paniers, de façon équitable et au bon prix.




- **IA (PyTorch)** : reconnaît les légumes sur une photo, le stock est saisi plus vite
- **Optimisation (OR-Tools)** : calcule la meilleure répartition
- **Contraintes** : prix cible, stock, variété, équité
- **Validation** : l'équipe valide, puis le client voit son prochain panier

IV

# Préparer et livrer

**Feuille de route – Tournée T1**
Livraison : mardi 12 mai

| Ordre | Point de dépôt | Simple | Familial | Œufs |
|---|---|---|---|---|
| 1 | Boulangerie du Centre | 8 | 5 | 3 |
| 2 | Mairie | 6 | 2 | 0 |
| 3 | CE Usine textile (réservé) | 10 | 4 | 2 |
| 4 | Pharmacie de la Gare | 3 | 1 | 1 |
| | **Total** | **27** | **12** | **6** |

*Pages suivantes : une page par dépôt (clients, téléphone, case signature)*

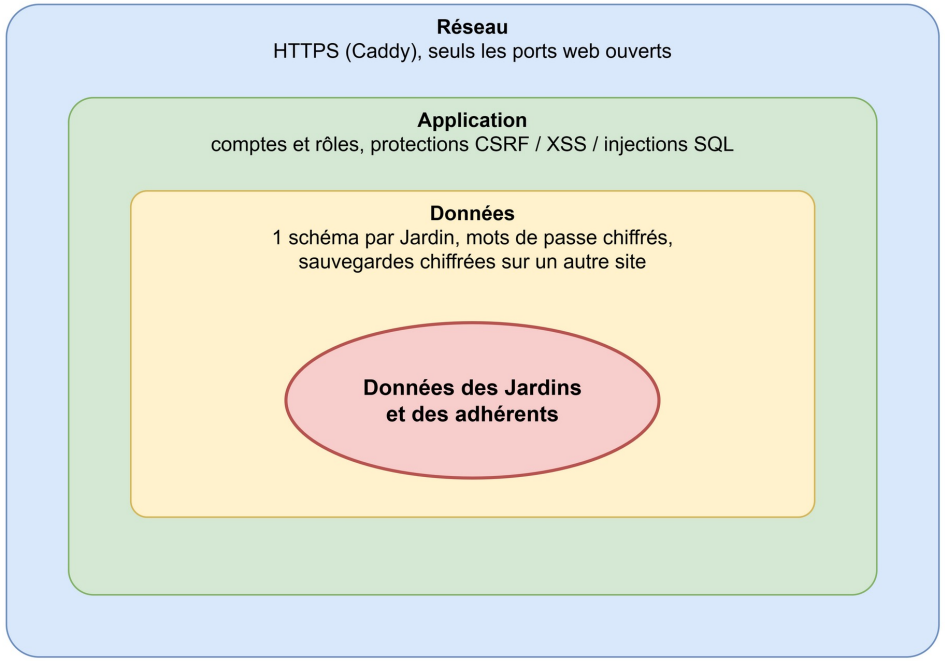
**✓ Livraison effectuée**

*sur la tablette du livreur*

Modèle HTML modifiable → PDF (WeasyPrint)

- **PDF automatiques** : feuille de préparation, étiquettes, feuille de route
- **Modèles modifiables** par un utilisateur averti (HTML → PDF avec WeasyPrint)
- **Sur tablette** : bouton « Livraison effectuée »
- **Exports CSV** : tableur, comptabilité
- **Prélèvements SEPA** : fichiers bancaires normalisés

# V Des données protégées




- **HTTPS** partout, géré automatiquement par Caddy
- **Comptes et rôles** : responsable, salarié, bénévole, client
- **Protections Django** : CSRF, XSS, injections SQL, mots de passe chiffrés
- **Sauvegardes** chiffrées chaque jour, stockées sur un autre site

## RGPD

Consentement, accès, rectification, suppression. Hébergement en France.

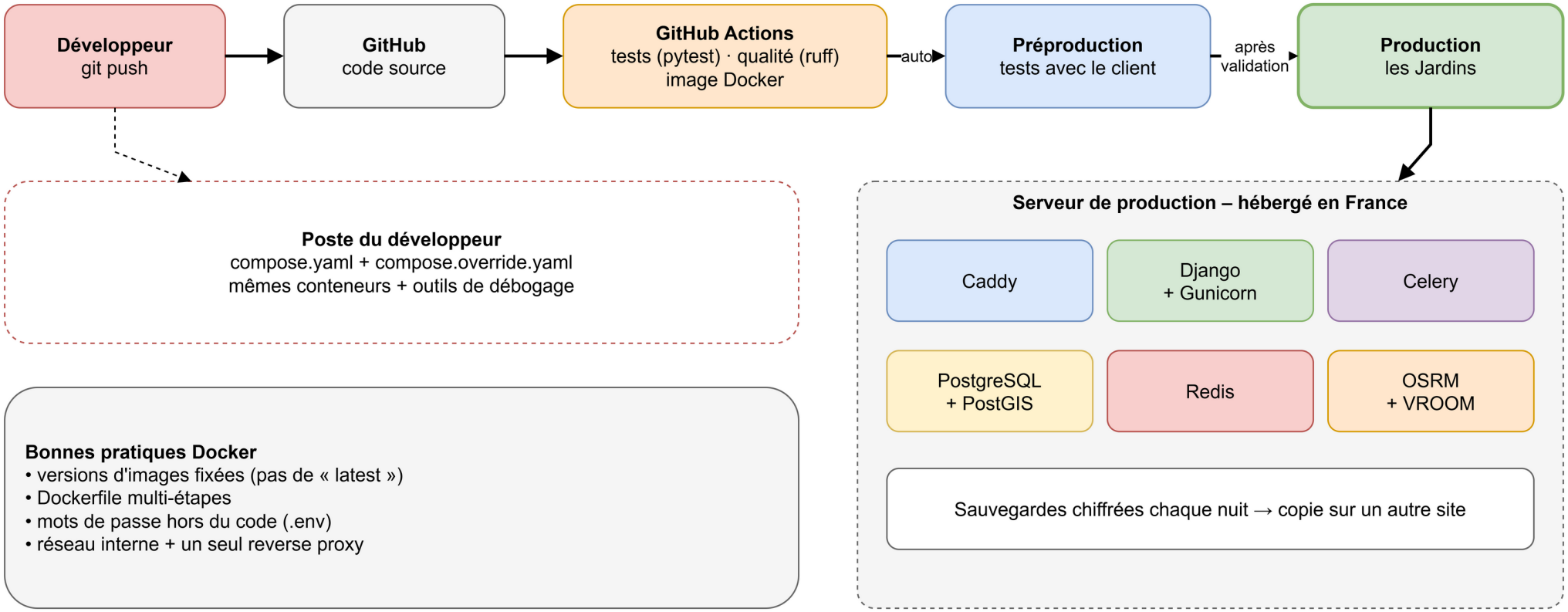
## Loi de finances

Factures non modifiables après validation, archivage 10 ans.

## SEPA

Mandats de prélèvement et fichiers bancaires normalisés.

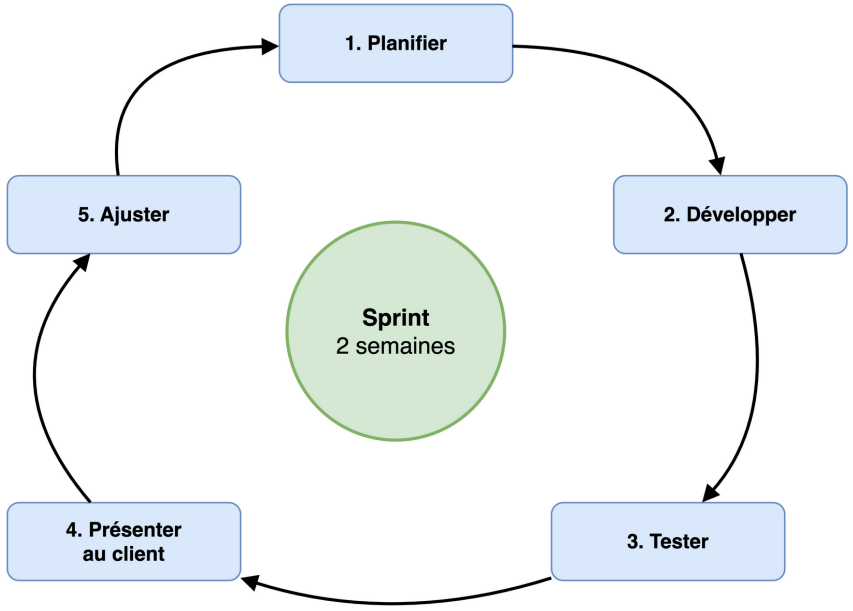
# De l'ordinateur du développeur aux Jardins




- **Docker** : chaque service dans son conteneur
- **3 environnements** : développement, préproduction, production
- **GitHub Actions** : tests et contrôle qualité automatiques
- **Mise en production** après validation en préproduction
- **Hébergé en France** : OVHcloud ou Scaleway

V

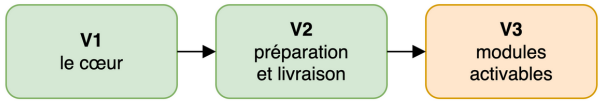
# Comment nous travaillons




**Outils**

**Trello** : tâches · **Clockify** : temps passé
**GitHub** : code, une branche par fonctionnalité, relecture avant fusion (Pull Request)

**Livraisons par étapes**




**Le binôme**

Chaque rendu est réalisé à deux. Pour chacun, un **pilote** organise le travail et vérifie les délais ; le suivi se fait dans Trello.

- **Démarche itérative** : sprints de 2 semaines
- **À chaque sprint** : une version utilisable présentée au client
- **Trello** : tâches · **Clockify** : temps passé
- **GitHub** : une branche par fonctionnalité, relecture avant fusion
- **Livraisons par étapes** : V1 le cœur, V2 préparation et livraison, V3 modules activables

SYNTHÈSE

# La solution en un coup d'œil

| Besoin | Technologie | Pourquoi |
|---|---|---|
| **Langage** | Python 3.13 | Simple, référence de l'IA |
| **Framework** | Django 5.2 LTS | Complet, admin intégrée, durable |
| **API** | Django REST Framework | Accès aux données pour tous les outils |
| **Interface** | Templates + HTMX + Bootstrap 5 | Simple, adaptée à tous les écrans |
| **Base de données** | PostgreSQL 17 + PostGIS | Robuste, géographique, 1 schéma par Jardin |
| **Carte** | Leaflet + OpenStreetMap + API Adresse | Libre et gratuit |
| **Itinéraire** | OSRM + VROOM | Auto-hébergé, sans coût à l'usage |
| **Calendrier** | rrule + holidays + FullCalendar | Calcul fiable, export vers l'agenda |
| **IA et optimisation** | PyTorch + OR-Tools + Mistral (Ollama) | Références du domaine, modèle libre |
| **Documents** | WeasyPrint | Modèles HTML modifiables |
| **Tâches de fond** | Celery + Redis | Pages toujours rapides |
| **Hébergement** | Docker + Caddy + GitHub Actions | Identique partout, HTTPS automatique |

# Notre proposition

## 100 %

open source

## 1 logiciel

pour tout le réseau,
chaque Jardin chez soi

## Simple

pour les équipes, et à reprendre pour les bénévoles

## Évolutif

des modules à activer selon les besoins

**Prochaine étape : la proposition financière**